

# TUGAS MANDIRI MODUL

## 7 (UCS)

Nama : Nuevalen Refitra Alswando  
NIM : 103072430008  
Kelas : IF-04-01

### SOAL 1

Sebuah kampus ingin membuat sistem pencarian rute antar gedung menggunakan algoritma Uniform Cost Search (UCS). Setiap gedung direpresentasikan sebagai node dalam graf, dan jarak antar gedung direpresentasikan sebagai bobot (cost).

Program harus memungkinkan pengguna untuk menentukan rute tercepat dari satu gedung ke gedung lainnya berdasarkan total jarak minimum.

### KETENTUAN PROGRAM :

1. Representasikan graf dalam bentuk dictionary adjacency list dengan format:

```
graph = {  
    'A': [('B', 4), ('C', 2)],  
    ...  
}
```

2. Program harus memiliki fitur Input node awal (start) dan Input node tujuan (goal)
3. Implementasikan algoritma Uniform Cost Search (UCS) menggunakan priority queue
4. Program harus bisa menampilkan Jalur terpendek dari start ke goal dan total cost dari jalur tersebut
5. Jika tidak ada jalur, tampilkan pesan : "Jalur tidak ditemukan"

```

1 import heapq
2 graph = {
3     'A': [('B', 4), ('C', 2)],
4     'B': [('A', 4), ('D', 5), ('E', 10)],
5     'C': [('A', 2), ('F', 3)],
6     'D': [('B', 5)],
7     'E': [('B', 10), ('F', 4), ('G', 6)],
8     'F': [('C', 3), ('E', 4), ('G', 2)],
9     'G': [('E', 6), ('F', 2)]
10 }
11 def ucs(graph, start, goal):
12     queue = [(0, start, [start])]
13     visited = set()
14
15     while queue:
16         cost, node, path = heapq.heappop(queue)
17         if node == goal:
18             return path, cost
19         if node not in visited:
20             visited.add(node)
21             for neighbor, weight in graph[node]:
22                 if neighbor not in visited:
23                     total_cost = cost + weight
24                     heapq.heappush(
25                         queue, (total_cost, neighbor, path + [neighbor]))
26
27     return None, None
28
29
30 start = input("Masukkan node awal : ").upper()
31 goal = input("Masukkan node tujuan : ").upper()
32 path, cost = ucs(graph, start, goal)
33
34 if path:
35     print("Jalur tercepat dari", start, "ke", goal, ":")
36     print(" -> ".join(path))
37     print("Total cost :", cost)
38
39 else:
40     print("Jalur tidak ditemukan")

```

```

(.venv) PS C:\Storage\Documents\Telkom University\Semester 4\Dasar Kecerdasan Artifisial\praktikum\modul07\mandiri> python soal1.py
Masukkan node awal : A
Masukkan node tujuan : G
Jalur tercepat dari A ke G :
A -> C -> F -> G
Total cost : 7

```

Pada soal 1, saya membuat program pencarian rute antar gedung kampus menggunakan algoritma Uniform Cost Search (UCS) dengan representasi graf berbentuk adjacency list, sehingga program dapat menentukan jalur tercepat dari node awal ke node tujuan berdasarkan total cost atau jarak minimum menggunakan priority queue.

## SOAL 2

Sebuah perusahaan logistik ingin mengembangkan sistem distribusi barang menggunakan algoritma Uniform Cost Search (UCS). Setiap kota direpresentasikan sebagai node, dan biaya pengiriman antar kota sebagai bobot edge.

Program harus dapat membantu menentukan rute pengiriman dengan biaya paling murah.

### **KETENTUAN PROGRAM :**

1. Gunakan graf berbobot (bebas, minimal 6 node)
2. Program yang dibuat harus memiliki fitur :
  - Menampilkan seluruh graf
  - Input kota asal dan kota tujuan
  - Menggunakan algoritma UCS untuk mencari rute
3. Tambahkan juga fitur tambahan sebagai berikut :
  - Menampilkan semua node yang dikunjungi selama proses pencarian
  - Menampilkan urutan ekspansi node
4. Output program harus mencakup :
  - Jalur terbaik
  - Total biaya
  - Urutan node yang dikunjungi

Pada soal 2, saya membuat program sistem distribusi barang menggunakan algoritma Uniform Cost Search (UCS) pada graf berbobot untuk mencari rute pengiriman dengan biaya paling murah antar kota, sekaligus menampilkan jalur terbaik, total biaya, node yang dikunjungi, serta urutan ekspansi node selama proses pencarian.

modul07 > mandiri > soal2.py > ...

```
1 import heapq
2 import networkx as nx
3 import matplotlib.pyplot as plt
4
5 def show_graph(G, pos=None, title=''):
6
7     if pos is None:
8         pos = nx.spring_layout(G)
9
10    nx.draw(
11        G,
12        pos,
13        with_labels=True,
14        node_color='red',
15        node_size=2000,
16        font_color='white',
17        font_weight='bold',
18        width=3
19    )
20
21    edge_labels = nx.get_edge_attributes(G, 'weight')
22    nx.draw_networkx_edge_labels(
23        G,
24        pos,
25        edge_labels=edge_labels,
26        font_color='blue',
27        font_weight='bold'
28    )
29
30    plt.title(title)
31    plt.show()
32
33 graph = {
34     'Surabaya': [('Malang', 2), ('Madiun', 5)],
35     'Malang': [('Surabaya', 2), ('Kediri', 4)],
36     'Madiun': [('Surabaya', 5), ('Kediri', 1), ('Solo', 6)],
37     'Kediri': [('Malang', 4), ('Madiun', 1), ('Yogyakarta', 7)],
38     'Solo': [('Madiun', 6), ('Yogyakarta', 3)],
39     'Yogyakarta': [('Kediri', 7), ('Solo', 3)]
40 }
41 G = nx.Graph()
42
43 for node in graph:
44     for neighbor, weight in graph[node]:
45         G.add_edge(node, neighbor, weight=weight)
46
```

```

47 pos = {
48     'Surabaya': (0, 2),
49     'Malang': (-2, 1),
50     'Madiun': (2, 1),
51     'Kediri': (0, 0),
52     'Solo': (3, -1),
53     'Yogyakarta': (1, -2)
54 }
55
56 show_graph(G, pos=pos, title='Graf Distribusi Barang')
57
58 def ucs(graph, start, goal):
59     queue = [(0, start, [start])]
60     visited = []
61     expansion = []
62
63     while queue:
64         cost, node, path = heapq.heappop(queue)
65         if node not in visited:
66             visited.append(node)
67             expansion.append(node)
68             if node == goal:
69                 return path, cost, visited, expansion
70             for neighbor, weight in graph[node]:
71                 if neighbor not in visited:
72                     total_cost = cost + weight
73                     heapq.heappush(
74                         queue,
75                         (total_cost, neighbor, path + [neighbor])
76                     )
77     return None, None, visited, expansion
78
79 start = input("Masukkan kota asal : ")
80 goal = input("Masukkan kota tujuan : ")
81
82 path, cost, visited, expansion = ucs(graph, start, goal)
83
84 if path:
85     print("\nJalur terbaik :")
86     print(" -> ".join(path))
87     print("\nTotal biaya :")
88     print(cost)
89     print("\nNode yang dikunjungi :")
90     print(visited)
91     print("\nUrutan ekspansi node :")
92     print(expansion)
93 else:
94     print("Jalur tidak ditemukan")

```

